

AI/BigData 인텐시브 과정

강사장철원

Section 0

코스소개

DAY1

DAY2

DAY3

DAY4

DAY5

데이터 분석을 위한 기초 수학

데이터 집계 및 시각화

DAY6

DAY7

DAY8

DAY9

DAY10

미니
프로젝트

확률분포, AB테스트

DAY11

DAY12

DAY13

DAY14

DAY15

머신러닝

DAY16

DAY17

DAY18

DAY19

DAY20

딥러닝

파이널 프로젝트

□ 파이토치를 활용한 분류

□ 파이토치를 활용한 회귀

AI/BigData 인텐시브 과정

Section 1. 딥러닝

Section 1-1. 파이토치를 활용한 분류

라이브러리 불러오기

In [1]: ▶

```
1 import pandas as pd
2
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from torch.utils.data import TensorDataset, random_split, DataLoader
```

데이터 프레임 다루기

파이토치

파이토치 신경망 모듈
파이토치 최적화 모듈

파이토치 데이터 처리

데이터를 딥러닝 학습용으로 가공

데이터를 텐서 자료구조로 가공

분류

```
1 device = 'cpu'
2 if torch.cuda.is_available():
3     device = 'cuda'
4
5 torch.manual_seed(777)
6 if device == 'cuda':
7     torch.cuda.manual_seed_all(777)
```

장치 설정

cuda 사용 가능하면 'cuda로 설정'

랜덤 시드 설정

cuda 랜덤 시드 설정

Section

파이토치를 활용한 분류

```
[3]: df = pd.read_csv("../data/iris.csv")
```

```
[4]: df
```

```
[4]:
```

	sepal_length	sepal_width	petal_length	petal_width	class
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

150 rows × 5 columns

분류

전체 행 선택

```
1 X = df.iloc[:, :-1].values
2 y = df.iloc[:, -1].values
```

마지막 열 선택

1	y
---	---

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2], dtype=int64)
```

1	X
---	---

(array) [[5.1, 3.5, 1.4, 0.2],
[4.9, 3. , 1.4, 0.2],
[4.7, 3.2, 1.3, 0.2],
[4.6, 3.1, 1.5, 0.2],
[5. , 3.6, 1.4, 0.2],
[5.4, 3.9, 1.7, 0.4],
[4.6, 3.4, 1.4, 0.3],
[5. , 3.4, 1.5, 0.2],
[4.4, 2.9, 1.4, 0.2],
[4.9, 3.1, 1.5, 0.1],
[5.4, 3.7, 1.5, 0.2],
[4.8, 3.4, 1.6, 0.2],
[4.8, 3. , 1.4, 0.1],
[4.3, 3. , 1.1, 0.1],
[5.8, 4. , 1.2, 0.2],
[5.7, 4.4, 1.5, 0.4],
[5.4, 3.9, 1.3, 0.4],
[5.1, 3.5, 1.4, 0.3],
[5.7, 3.8, 1.7, 0.3].

Section

파이토치를 활용한 분류

```
In [12]: 1 type(df.iloc[:, :-1])
```

```
Out[12]: pandas.core.frame.DataFrame
```

```
In [11]: 1 df.iloc[:, :-1]
```

```
Out[11]:
```

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

```
In [13]: 1 type(df.iloc[:, :-1].values)
```

```
Out[13]: numpy.ndarray
```

```
In [14]: 1 df.iloc[:, :-1].values
```

```
Out[14]: array([[5.1, 3.5, 1.4, 0.2],  
                [4.9, 3. , 1.4, 0.2],  
                [4.7, 3.2, 1.3, 0.2],  
                [4.6, 3.1, 1.5, 0.2],  
                [5. , 3.6, 1.4, 0.2],  
                [5.4, 3.9, 1.7, 0.4],  
                [4.6, 3.4, 1.4, 0.3],  
                [5. , 3.4, 1.5, 0.2],  
                [4.4, 2.9, 1.4, 0.2],  
                [4.9, 3.1, 1.5, 0.1],  
                [5.4, 3.7, 1.5, 0.2],  
                [4.8, 3.4, 1.6, 0.2],  
                [4.8, 3. , 1.4, 0.1],  
                [4.3, 3. , 1.1, 0.1],
```

분류

실수 형태로 변경

```
1 X = torch.tensor(X, dtype=torch.float32).to(device)
2 y = torch.tensor(y, dtype=torch.long).to(device)
```

정수 형태로 변경

1 y

```
tensor([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2])
```

정수

tensor 형식이라는
것을 알 수 있다.

1 X

```
tensor([[5.1000, 3.5000, 1.4000, 0.2000],
        [4.9000, 3.0000, 1.4000, 0.2000],
        [4.7000, 3.2000, 1.3000, 0.2000],
        [4.6000, 3.1000, 1.5000, 0.2000],
        [5.0000, 3.6000, 1.4000, 0.2000],
        [5.4000, 3.9000, 1.7000, 0.4000],
        [4.6000, 3.4000, 1.4000, 0.3000],
        [5.0000, 3.4000, 1.5000, 0.2000],
        [4.4000, 2.9000, 1.4000, 0.2000],
        [4.9000, 3.1000, 1.5000, 0.1000],
        [5.4000, 3.7000, 1.5000, 0.2000],
        [4.8000, 3.4000, 1.6000, 0.2000],
        [4.8000, 3.0000, 1.4000, 0.1000],
        [4.3000, 3.0000, 1.1000, 0.1000],
        [5.8000, 4.0000, 1.2000, 0.2000],
        [5.7000, 4.4000, 1.5000, 0.4000],
        [5.4000, 3.9000, 1.3000, 0.4000],
        [5.1000, 3.5000, 1.4000, 0.3000],
        [5.7000, 3.8000, 1.7000, 0.3000],
```

실수

피처와 라벨을 묶어줌

```
1 dataset = TensorDataset(X, y)
```

```
1 n = len(dataset)
2 ratio_tn = 0.8
3 n_tn = int(n*ratio_tn)
4 n_te = n - n_tn
```

```
1 print("전체 데이터 개수:", n)
2 print("트레이닝 데이터 비율:", ratio_tn)
3 print("트레이닝 데이터 개수:", n_tn)
4 print("테스트 데이터 개수:", n_te)
```

전체 데이터 개수: 150
 트레이닝 데이터 비율: 0.8
 트레이닝 데이터 개수: 120
 테스트 데이터 개수: 30

데이터 셋 확인

```
In [22]: 1 dataset = TensorDataset(X, y)
```

```
In [26]: 1 for i in range(0,10):
          2     print(dataset[i])
```

```
(tensor([5.1000, 3.5000, 1.4000, 0.2000]), tensor(0))
(tensor([4.9000, 3.0000, 1.4000, 0.2000]), tensor(0))
(tensor([4.7000, 3.2000, 1.3000, 0.2000]), tensor(0))
(tensor([4.6000, 3.1000, 1.5000, 0.2000]), tensor(0))
(tensor([5.0000, 3.6000, 1.4000, 0.2000]), tensor(0))
(tensor([5.4000, 3.9000, 1.7000, 0.4000]), tensor(0))
(tensor([4.6000, 3.4000, 1.4000, 0.3000]), tensor(0))
(tensor([5.0000, 3.4000, 1.5000, 0.2000]), tensor(0))
(tensor([4.4000, 2.9000, 1.4000, 0.2000]), tensor(0))
(tensor([4.9000, 3.1000, 1.5000, 0.1000]), tensor(0))
```

Section

파이토치를 활용한 분류

X				y
F1	F2	F3	F4	T

Pandas DataFrame

(tensor(), tensor())

(tensor(), tensor())

(tensor(), tensor())

(tensor(), tensor())

(tensor(), tensor())

(tensor(), tensor())

(tensor(), tensor())

(tensor(), tensor())

(tensor(), tensor())

(tensor(), tensor())

Pytorch TensorDataset

트레이닝 데이터와 테스트 데이터를 분리

```
[16]: train_data, test_data = random_split(dataset, [n_tn, n_te])
```

```
[17]: for i in range(0,3):  
        feature, target = train_data[i]  
        print("Feature: ", feature)  
        print("Target: ", target)  
        print()
```

Feature: tensor([5.0000, 3.2000, 1.2000, 0.2000])

Target: tensor(0)

Feature: tensor([7.3000, 2.9000, 6.3000, 1.8000])

Target: tensor(2)

Feature: tensor([7.2000, 3.0000, 5.8000, 1.6000])

Target: tensor(2)

DataLoader 함수

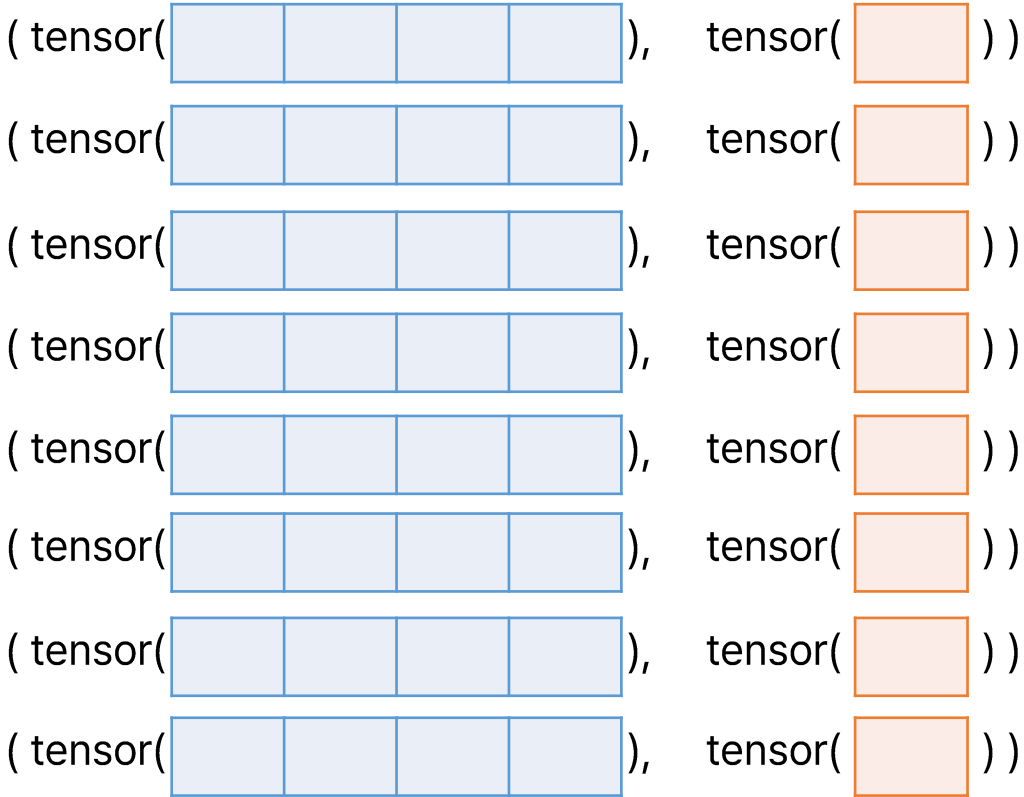
- 데이터를 학습 시킬 때 모형에 모든 데이터를 한꺼번에 넣는 것이 아니라 미니 배치 형식으로 넣을 필요가 있는데,
- 데이터를 모형에 넣을 때 미니 배치 형식으로 데이터를 쪼개서 넣을 수 있도록 DataLoader 함수가 도와줌

```
1 n_batch = 16
2 train_loader = DataLoader(train_data, batch_size=n_batch, shuffle=True)
```

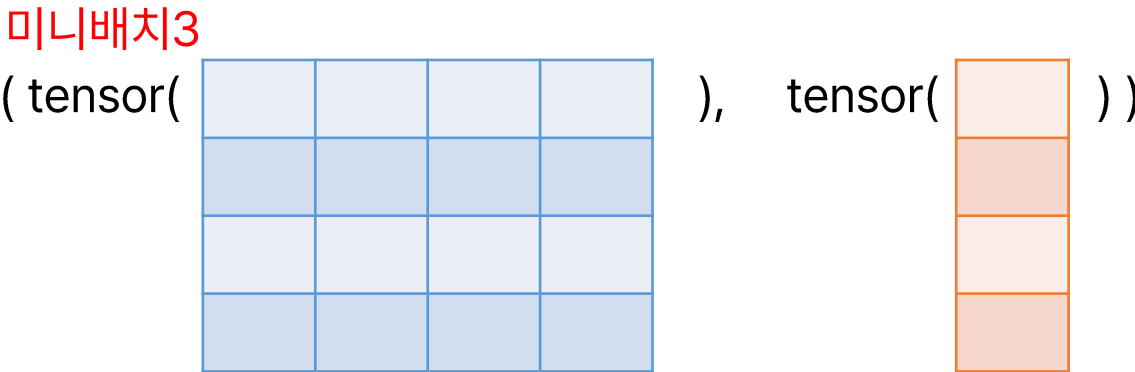
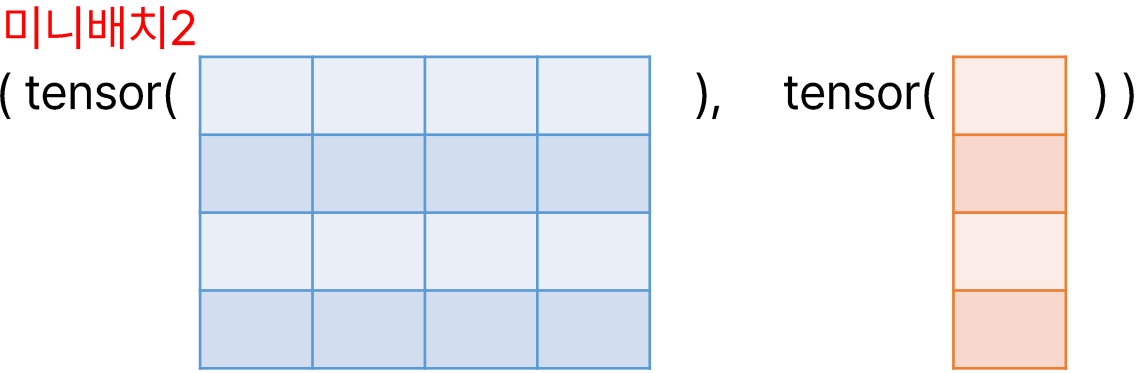
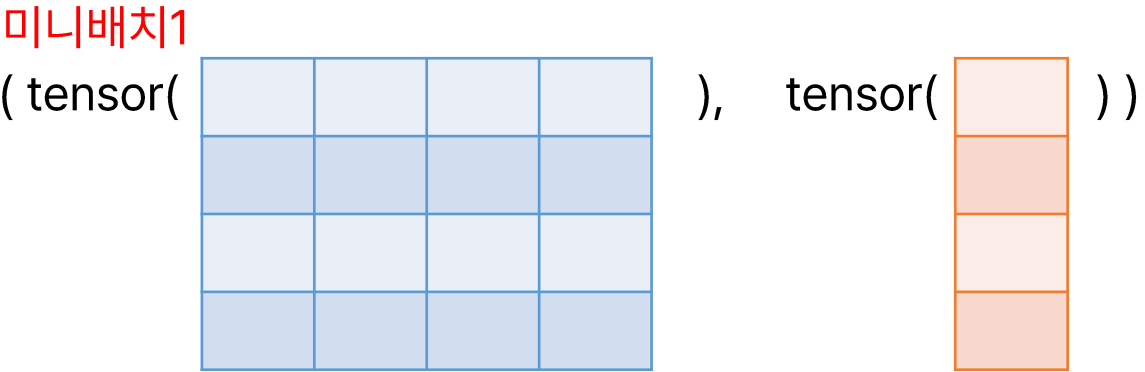
미니 배치 사이즈

모든 미니 배치 순회(iteration)
이후 데이터 섞임

Section
파이토치를 활용한 분류



Pytorch TensorDataset



DataLoader

```

batch_count = 0
for minibatch in train_loader:
    features, targets = minibatch
    print("Feature:", features)
    print("Target:", targets)
    print()

    batch_count += 1
    if batch_count >= 2:
        break

```

미니 배치1

```

Feature: tensor([[5.1000, 3.4000, 1.5000, 0.2000],
                [6.4000, 2.9000, 4.3000, 1.3000],
                [7.2000, 3.0000, 5.8000, 1.6000],
                [6.3000, 3.3000, 6.0000, 2.5000],
                [6.3000, 2.7000, 4.9000, 1.8000],
                [6.7000, 3.1000, 4.4000, 1.4000],
                [5.5000, 2.5000, 4.0000, 1.3000],
                [5.7000, 3.0000, 4.2000, 1.2000],
                [5.1000, 3.8000, 1.6000, 0.2000],
                [6.4000, 2.7000, 5.3000, 1.9000],
                [6.5000, 3.0000, 5.5000, 1.8000],
                [5.0000, 3.4000, 1.6000, 0.4000],
                [4.6000, 3.4000, 1.4000, 0.3000],
                [6.9000, 3.1000, 5.4000, 2.1000],
                [6.7000, 3.1000, 5.6000, 2.4000],
                [5.5000, 2.6000, 4.4000, 1.2000]])
Target: tensor([0, 1, 2, 2, 2, 1, 1, 1, 0, 2, 2, 0, 0, 2, 2, 1])

```

미니 배치2

```

Feature: tensor([[5.8000, 2.7000, 3.9000, 1.2000],
                [4.9000, 3.1000, 1.5000, 0.2000],
                [4.7000, 3.2000, 1.6000, 0.2000],
                [6.1000, 2.9000, 4.7000, 1.4000],
                [6.8000, 3.0000, 5.5000, 2.1000],
                [7.3000, 2.9000, 6.3000, 1.8000],
                [5.1000, 2.5000, 3.0000, 1.1000],
                [5.0000, 3.0000, 1.6000, 0.2000],
                [5.9000, 3.2000, 4.8000, 1.8000],
                [4.9000, 3.0000, 1.4000, 0.2000],
                [5.0000, 3.5000, 1.3000, 0.3000],
                [6.2000, 2.8000, 4.8000, 1.8000],
                [4.5000, 2.3000, 1.3000, 0.3000],
                [6.4000, 3.2000, 5.3000, 2.3000],
                [5.6000, 2.7000, 4.2000, 1.3000],
                [6.8000, 3.2000, 5.9000, 2.3000]])
Target: tensor([1, 0, 0, 1, 2, 2, 1, 0, 1, 0, 0, 2, 0, 2, 1, 2])

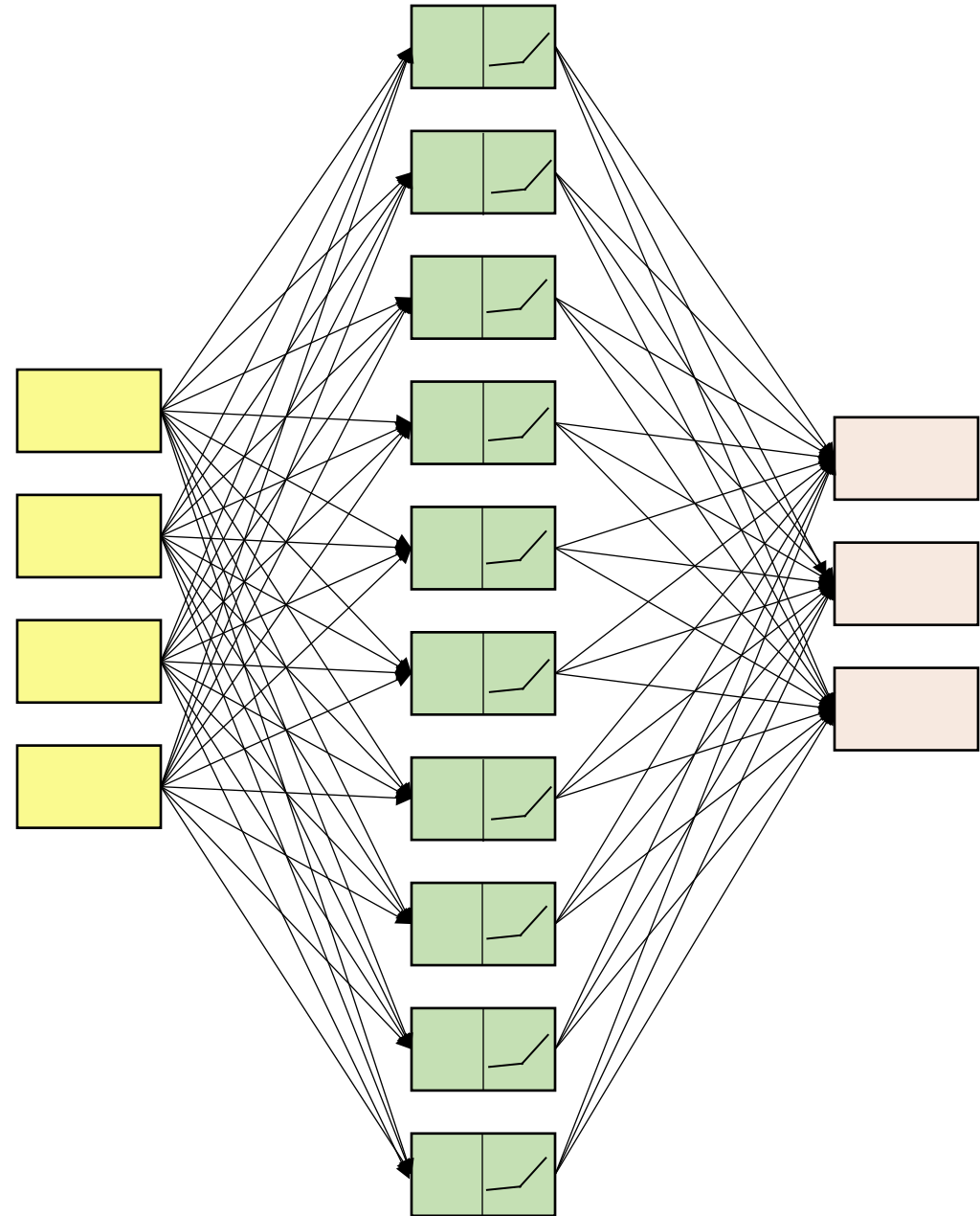
```


분류

```
[20]: # hyperparameters
learning_rate = 0.01
epochs = 1000
n_feature = len(train_data[0][0])
print(n_feature)
```

4

```
[20]: class Classifier(nn.Module):  
    def __init__(self, n_feature):  
        super().__init__()  
        self.fc1 = nn.Linear(n_feature, 10)  
        self.leakyrelu = nn.LeakyReLU()  
        self.fc2 = nn.Linear(10, 3)  
  
    def forward(self, x):  
        x = self.fc1(x)  
        x = self.leakyrelu(x)  
        x = self.fc2(x)  
        return x
```



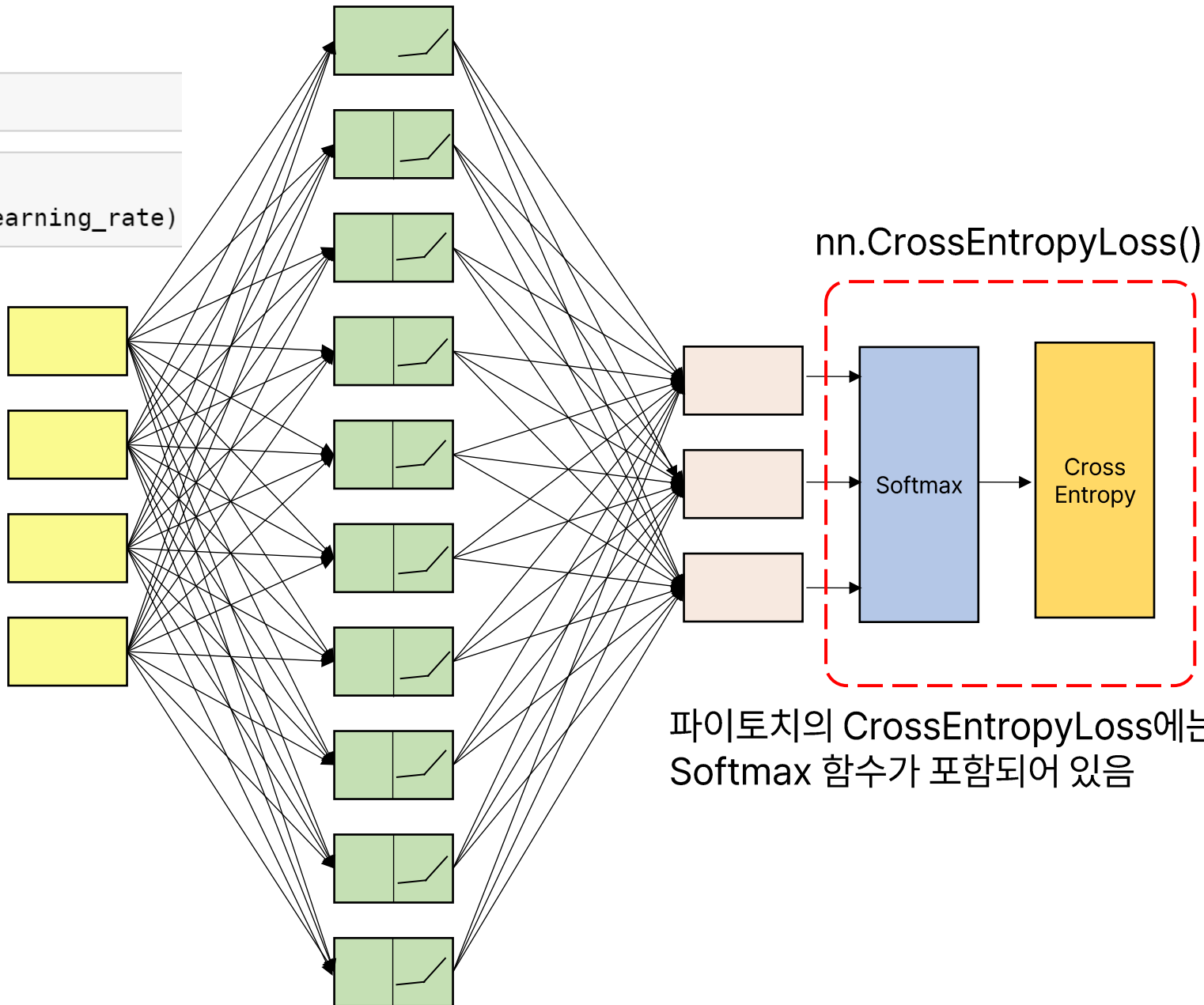
Section

파이토치를 활용한 분류

```
1 model = Classifier(n_feature).to(device)
```

```
1 criterion = nn.CrossEntropyLoss().to(device)  
2 optimizer = optim.SGD(model.parameters(), lr=learning_rate)
```

파이토치의 CrossEntropyLoss에는
Softmax 함수가 포함되어 있음



파이토치의 CrossEntropyLoss에는
Softmax 함수가 포함되어 있음

Section

파이토치를 활용한 분류

```
1 total_batch = len(train_loader)
2 model.train()
3 for epoch in range(epochs):
4     avg_cost = 0
5     for X_tn, y_tn in train_loader:
6
7         X_tn = X_tn.to(device)
8         y_tn = y_tn.to(device)
9
10        optimizer.zero_grad()
11        y_hat = model.forward(X_tn)
12        cost = criterion(y_hat, y_tn)
13        cost.backward()
14        optimizer.step()
15
16        avg_cost += cost/total_batch
17
18    if (epoch+1) % 100 == 0:
19        print('Epoch [%d/%d], Cost: %.4f' % (epoch+1, epochs, avg_cost))
```

모델을 학습 모드로 설정

model.train() – 학습(training) 모드

- 학습 모드에서는 오버피팅 방지를 위해 dropout layer 활성화

model.eval() – 추론(inference) 모드

- 추론이란 새롭게 입력되는 피쳐 데이터를 활용해 예측을 수행하는 과정

- 추론 모드에서는 일관된 결과를 얻기 위해 dropout layer 비활성화

```
Epoch [100/1000], Cost: 0.2826
Epoch [200/1000], Cost: 0.1438
Epoch [300/1000], Cost: 0.1313
Epoch [400/1000], Cost: 0.0989
Epoch [500/1000], Cost: 0.0925
Epoch [600/1000], Cost: 0.0843
Epoch [700/1000], Cost: 0.0785
Epoch [800/1000], Cost: 0.0743
Epoch [900/1000], Cost: 0.0992
Epoch [1000/1000], Cost: 0.0784
```

Section

파이토치를 활용한 분류

```
1 total_batch = len(train_loader)
2 model.train()
3 for epoch in range(epochs):
4     avg_cost = 0
5     for X_tn, y_tn in train_loader:
6
7         X_tn = X_tn.to(device)
8         y_tn = y_tn.to(device)
9
10        optimizer.zero_grad()
11        y_hat = model.forward(X_tn)
12        cost = criterion(y_hat, y_tn)
13        cost.backward()
14        optimizer.step()
15
16        avg_cost += cost/total_batch
17
18    if (epoch+1) % 100 == 0:
19        print('Epoch [%d/%d], Cost: %.4f' % (epoch+1, epochs, avg_cost))
```

그래디언트 초기화

(미니배치가 진행될 때마다 그래디언트는 계속 초기화 됨,
그래디언트 누적시 최적 방향으로 업데이트 되지 않을 수도 있음)

순전파 연산 수행 -> 입력 데이터를 모델에 전달하고 예측값을 산출

loss 계산

그래디언트 계산

앞서 계산된 그래디언트를 사용해 모델 파라미터 업데이트
(에포크가 진행될 때 파라미터는 초기화 되지 않고 계속 업데이트됨)

```
Epoch [100/1000], Cost: 0.2826
Epoch [200/1000], Cost: 0.1438
Epoch [300/1000], Cost: 0.1313
Epoch [400/1000], Cost: 0.0989
Epoch [500/1000], Cost: 0.0925
Epoch [600/1000], Cost: 0.0843
Epoch [700/1000], Cost: 0.0785
Epoch [800/1000], Cost: 0.0743
Epoch [900/1000], Cost: 0.0992
Epoch [1000/1000], Cost: 0.0784
```

분류

1	y_hat
---	-------

```
tensor([[ -11.8435,    2.2322,    9.3875],  
        [  -8.5642,    4.1462,    5.0239],  
        [  -9.4227,    2.6434,    6.8071],  
        [  -1.4509,    4.6262,   -2.0076],  
        [  -1.0946,    4.8309,   -2.5748],  
        [ 13.7059,    7.9695,  -19.0682],  
        [-11.3740,    2.9374,    8.4218],  
        [  -4.2067,    3.6314,    1.2739]], grad_fn=<AddmmBackward0>)
```

1	y_tn
---	------

```
tensor([2, 2, 2, 1, 1, 0, 2, 1])
```

분류

```
1 X_te = []
2 y_te = []
3
4 for row in test_data:
5     feature, target = row
6     X_te.append(feature)
7     y_te.append(target)
```

```
1 X_te = torch.stack(X_te)
2 y_te = torch.stack(y_te)
```

분류

```
model.eval()
with torch.no_grad():
    prediction = model.forward(X_te)
    correct_prediction = torch.argmax(prediction, 1) == y_te
    accuracy = correct_prediction.float().mean()
    print('Accuracy:', accuracy.item())
```

Accuracy: 0.9666666388511658

y_te

```
tensor([2, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 2, 0, 2,
        0, 0, 0, 2, 0, 0])
```

torch.argmax(prediction, 1)

```
tensor([2, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 2, 1, 1, 1, 1, 0, 1, 0, 1, 1, 2, 0, 2,
        0, 0, 0, 2, 0, 0])
```


분류



```
1 PATH = "./iris_model01.pt"  
2 torch.save(model.state_dict(), PATH)
```

분류

```
PATH = "./iris_model01.pt"
```

```
# Load the saved model
```

```
loaded_model = Classifier(n_feature).to(device)
```

```
loaded_model.load_state_dict(torch.load(PATH))
```

<All keys matched successfully>

```
loaded_model.eval()
```

```
with torch.no_grad():
```

```
    prediction = loaded_model(X_te)
```

```
    correct_prediction = torch.argmax(prediction, 1) == y_te
```

```
    accuracy = correct_prediction.float().mean()
```

```
    print('Accuracy:', accuracy.item())
```

Accuracy: 0.9666666388511658

```

class Classifier(nn.Module):
    def __init__(self, n_feature):
        super().__init__()
        self.fc1 = nn.Linear(n_feature, 10)
        self.bn1 = nn.BatchNorm1d(num_features=10)
        self.leakyrelu = nn.LeakyReLU()
        self.dropout = nn.Dropout(p=0.2)
        self.fc2 = nn.Linear(10, 10)
        self.bn2 = nn.BatchNorm1d(num_features=10)
        self.fc3 = nn.Linear(10, 3)
        self.bn3 = nn.BatchNorm1d(num_features=3)

    def forward(self, x):
        x = self.fc1(x)
        x = self.bn1(x)
        x = self.leakyrelu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        x = self.bn2(x)
        x = self.leakyrelu(x)
        x = self.dropout(x)
        x = self.fc3(x)
        x = self.bn3(x)
        return x

```

```

class Classifier(nn.Module):
    def __init__(self, n_feature):
        super().__init__()
        self.layer1 = nn.Sequential(
            nn.Linear(n_feature, 10),
            nn.BatchNorm1d(num_features=10),
            nn.LeakyReLU(),
            nn.Dropout(p=0.2)
        )

        self.layer2 = nn.Sequential(
            nn.Linear(10, 10),
            nn.BatchNorm1d(num_features=10),
            nn.LeakyReLU(),
            nn.Dropout(p=0.2)
        )

        self.layer3 = nn.Sequential(
            nn.Linear(10, 3),
            nn.BatchNorm1d(num_features=3)
        )

    def forward(self, x):
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        return x

```

AI/BigData 인텐시브 과정

Section 1. 딥러닝

Section 1-2. 파이토치를 활용한 회귀

```
1 import pandas as pd
2
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from torch.utils.data import TensorDataset, random_split, DataLoader
```

```
1 device = 'cpu'
2 if torch.cuda.is_available():
3     device = 'cuda'
4
5 torch.manual_seed(777)
6 if device == 'cuda':
7     torch.cuda.manual_seed_all(777)
```

Section

파이토치를 활용한 회귀

```
[3]: df = pd.read_csv("../data/house_prices.csv")
```

```
[4]: df.head(3)
```

```
[4]:
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7

```
[5]: X = df.iloc[:, :-1].values  
y = df.iloc[:, -1].values
```

```
[6]: X = torch.tensor(X, dtype=torch.float32).to(device)  
y = torch.tensor(y, dtype=torch.float32).to(device)
```

Section

파이토치를 활용한 회귀

```
1 dataset = TensorDataset(X, y)
```

```
1 n = len(dataset)
2 ratio_tn = 0.8
3 n_tn = int(n*ratio_tn)
4 n_te = n - n_tn
```

```
1 print("전체 데이터 개수:", n)
2 print("트레이닝 데이터 비율:", ratio_tn)
3 print("트레이닝 데이터 개수:", n_tn)
4 print("테스트 데이터 개수:", n_te)
```

전체 데이터 개수: 506

트레이닝 데이터 비율: 0.8

트레이닝 데이터 개수: 404

테스트 데이터 개수: 102

```
1 train_data, test_data = random_split(dataset, [n_tn, n_te])
```

```
1 for i in range(0,3):  
2     feature, target = train_data[i]  
3     print("Input: ", feature)  
4     print("Label: ", target)  
5     print()
```

Input: tensor([3.4940e-01, 0.0000e+00, 9.9000e+00, 0.0000e+00, 5.4400e-01, 5.9720e+00,
7.6700e+01, 3.1025e+00, 4.0000e+00, 3.0400e+02, 1.8400e+01, 3.9624e+02,
9.9700e+00])

Label: tensor(20.3000)

Input: tensor([3.5350, 0.0000, 19.5800, 1.0000, 0.8710, 6.1520, 82.6000,
1.7455, 5.0000, 403.0000, 14.7000, 88.0100, 15.0200])

Label: tensor(15.6000)

Input: tensor([5.0830e-02, 0.0000e+00, 5.1900e+00, 0.0000e+00, 5.1500e-01, 6.3160e+00,
3.8100e+01, 6.4584e+00, 5.0000e+00, 2.2400e+02, 2.0200e+01, 3.8971e+02,
5.6800e+00])

Label: tensor(22.2000)

회귀

```

1 batch_size = 16
2 train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True)

```

```

1 batch_count = 0
2 for batch in train_loader:
3     features, labels = batch
4     print("Features:", features)
5     print("Labels:", labels)
6     print()
7
8     batch_count += 1
9     if batch_count >= 2:
10        break

```

```

Features: tensor([[2.4522e-01, 0.0000e+00, 9.9000e+00, 0.0000e+00, 5.4400e-01, 5.7820e+00,
                  7.1700e+01, 4.0317e+00, 4.0000e+00, 3.0400e+02, 1.8400e+01, 3.9690e+02,
                  1.5940e+01],
                 [3.7662e+01, 0.0000e+00, 1.8100e+01, 0.0000e+00, 6.7900e-01, 6.2020e+00,
                  7.8700e+01, 1.8629e+00, 2.4000e+01, 6.6600e+02, 2.0200e+01, 1.8820e+01,
                  1.4520e+01],
                 [1.8159e-01, 0.0000e+00, 7.3800e+00, 0.0000e+00, 4.9300e-01, 6.3760e+00,
                  5.4300e+01, 4.5404e+00, 5.0000e+00, 2.8700e+02, 1.9600e+01, 3.9690e+02,
                  6.8700e+00],
                 [1.5876e-01, 0.0000e+00, 1.0810e+01, 0.0000e+00, 4.1300e-01, 5.9610e+00,
                  1.7500e+01, 5.2873e+00, 4.0000e+00, 3.0500e+02, 1.9200e+01, 3.7694e+02,
                  9.8800e+00],
                 [2.7957e-01, 0.0000e+00, 9.6900e+00, 0.0000e+00, 5.8500e-01, 5.9260e+00,
                  4.2600e+01, 2.3817e+00, 6.0000e+00, 3.9100e+02, 1.9200e+01, 3.9690e+02,
                  1.3590e+01],
                 [2.2424e+00, 0.0000e+00, 1.9580e+01, 0.0000e+00, 6.0500e-01, 5.8540e+00,
                  9.1800e+01, 2.4220e+00, 5.0000e+00, 4.0300e+02, 1.4700e+01, 3.9511e+02,
                  1.1640e+01],
                 [4.4178e-01, 0.0000e+00, 6.2000e+00, 0.0000e+00, 5.0400e-01, 6.5520e+00,

```

회귀

```
[14]: # hyperparameters
learning_rate = 0.01
epochs = 1000
n_feature = len(train_data[0][0])
print(n_feature)
```

13

```
[15]: class Regressor(nn.Module):
    def __init__(self, n_feature):
        super().__init__()
        self.fc1 = nn.Linear(n_feature, 30)
        self.bn1 = torch.nn.BatchNorm1d(30)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(p=0.3)
        self.fc2 = nn.Linear(30, 1)

    def forward(self, x):
        x = self.fc1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

회귀

```
1 model = Regressor(n_feature).to(device)
```

```
1 criterion = nn.MSELoss().to(device)
```

```
2 optimizer = optim.SGD(model.parameters(), lr=learning_rate)
```

Section

파이토치를 활용한 회귀

회귀

1 y_hat

tensor([41.1004, 22.4183, 10.6586, 20.2157], grad_fn=<SqueezeBackward0>)

1 y_tn

tensor([35.4000, 21.4000, 10.8000, 21.4000])

```
1 total_batch = len(train_loader)
2 model.train()      # set the model to evaluation mode (dropout=False)
3 for epoch in range(epochs):
4     avg_cost = 0
5     for X_tn, y_tn in train_loader:
6
7         X_tn = X_tn.to(device)
8         y_tn = y_tn.to(device)
9
10        optimizer.zero_grad()
11        y_hat = model.forward(X_tn)
12        y_hat = y_hat.squeeze()
13        cost = criterion(y_hat, y_tn)
14        cost.backward()
15        optimizer.step()
16
17        avg_cost += cost/total_batch
18
19    if (epoch+1) % 100 == 0:
20        print('Epoch [%d/%d], Cost: %.4f' % (epoch+1, epochs, avg_cost))
```

```
Epoch [100/1000], Cost: 64.0577
Epoch [200/1000], Cost: 44.8954
Epoch [300/1000], Cost: 42.3605
Epoch [400/1000], Cost: 40.8707
Epoch [500/1000], Cost: 35.5501
Epoch [600/1000], Cost: 40.1490
Epoch [700/1000], Cost: 48.9997
Epoch [800/1000], Cost: 34.7151
Epoch [900/1000], Cost: 46.8158
Epoch [1000/1000], Cost: 49.4516
```

Section

파이토치를 활용한 회귀

```
X_te = []
y_te = []

for row in test_data:
    feature, target = row
    X_te.append(feature)
    y_te.append(target)
```

```
X_te = torch.stack(X_te)
y_te = torch.stack(y_te)
```

```
model.eval()
with torch.no_grad():
    prediction = model.forward(X_te)
    prediction = prediction.squeeze()
    mse = criterion(prediction, y_te)
    print('MSE:', mse.item())
```

MSE: 24.259990692138672

1	y_te
tensor([	32.0000, 28.6000, 19.4000, 14.9000, 32.5000, 13.8000, 19.0000, 24.3000,
	24.2000, 22.5000, 11.0000, 9.5000, 17.1000, 31.6000, 25.0000, 15.0000,
	22.6000, 44.0000, 22.2000, 21.2000, 14.6000, 19.1000, 21.1000, 14.3000,
	21.2000, 37.6000, 7.0000, 20.6000, 24.0000, 31.5000, 20.6000, 19.9000,
	12.7000, 13.8000, 13.4000, 32.7000, 24.8000, 14.0000, 6.3000, 33.1000,
	18.6000, 13.5000, 18.9000, 20.3000, 22.3000, 35.1000, 30.3000, 20.6000,
	18.5000, 18.0000, 14.5000, 30.1000, 33.0000, 23.9000, 17.0000, 20.6000,
	16.1000, 24.8000, 20.6000, 24.4000, 8.3000, 15.4000, 24.5000, 18.4000,
	17.1000, 20.0000, 23.1000, 23.3000, 16.0000, 26.2000, 29.0000, 16.6000,
	14.1000, 21.0000, 11.3000, 13.1000, 19.2000, 13.0000, 20.1000, 14.4000,
	13.3000, 28.4000, 31.2000, 13.6000, 13.1000, 21.7000, 19.3000, 19.1000,
	24.7000, 36.2000, 29.0000, 41.3000, 50.0000, 15.6000, 19.0000, 13.8000,
	32.9000, 23.2000, 24.5000, 17.8000, 17.7000, 14.4000])

1	prediction
tensor([	32.4438, 28.9859, 22.9798, 20.5157, 36.7267, 11.2802, 26.1733, 25.8063,
	25.9787, 24.2518, 14.0262, 11.9903, 21.7261, 38.3983, 25.3717, 21.1184,
	24.0966, 38.9945, 28.2562, 23.1429, 16.7114, 18.1882, 26.7558, 20.0535,
	26.0049, 37.3060, 16.7819, 25.0748, 34.2346, 34.5960, 21.9404, 20.9795,
	21.2889, 15.5225, 12.6721, 33.0695, 27.1797, 17.7670, 15.0872, 32.9651,
	25.6486, 18.9715, 26.4144, 24.5985, 27.7826, 36.5775, 31.7848, 25.3591,
	27.0709, 21.4062, 18.7779, 36.5577, 28.8338, 31.1355, 23.6747, 28.0085,
	23.1542, 31.2377, 26.3906, 24.1553, 11.1552, 19.9007, 32.4964, 23.0490,
	19.6046, 23.4650, 25.1896, 30.4564, 23.5458, 27.8956, 33.5401, 22.0150,
	16.8340, 23.3623, 18.7067, 21.0818, 26.8575, 17.9833, 25.7928, 13.3209,
	16.7404, 31.1423, 32.6077, 19.3506, 21.6410, 22.9496, 22.8895, 21.9779,
	24.6229, 32.5972, 31.0383, 35.4527, 39.7746, 19.5992, 19.1996, 23.0559,
	32.9237, 19.8311, 25.7586, 16.9531, 23.5198, 16.4486])

모델 저장

```
▶ 1 PATH = "./boston01.pt"
   2 torch.save(model.state_dict(), PATH)
```

```
PATH = "./boston01.pt"
```

```
# Load the saved model
```

```
loaded_model = Regressor(n_feature).to(device)
loaded_model.load_state_dict(torch.load(PATH))
```

```
<All keys matched successfully>
```

```
loaded_model.eval()
with torch.no_grad():
    prediction = loaded_model.forward(X_te)
    prediction = prediction.squeeze()
    mse = criterion(prediction, y_te)
    print('MSE:', mse.item())
```

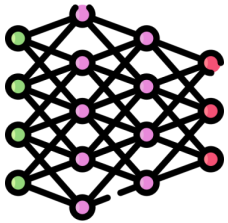
```
MSE: 24.259990692138672
```

AI/BigData 인텐시브 과정

Section 1. 딥러닝

Section 1-3. 여러가지 딥러닝 모형

딥러닝의 종류



딥러닝 기초
분류 / 회귀

합성곱 신경망(**CNN**, Convolutional Neural Network), 1989년 등장

순환 신경망(**RNN**, Recurrent Neural Network), 1986년 등장

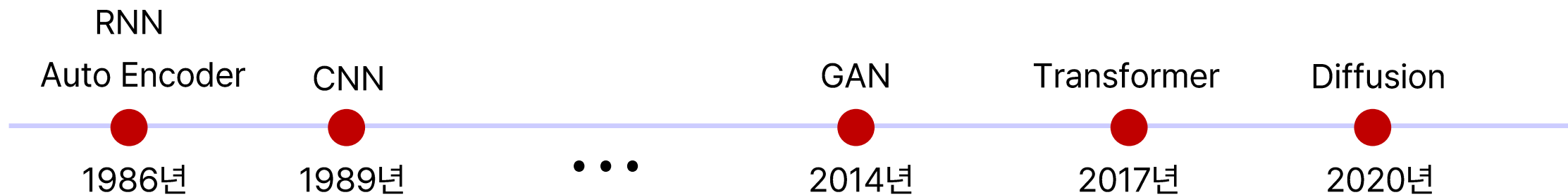
오토인코더(Auto Encoder), 1986년 등장

적대적 생성 신경망(Generative Adversarial Network), 2014년 등장

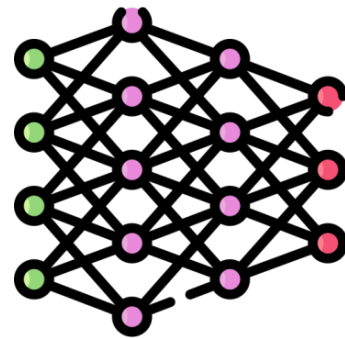
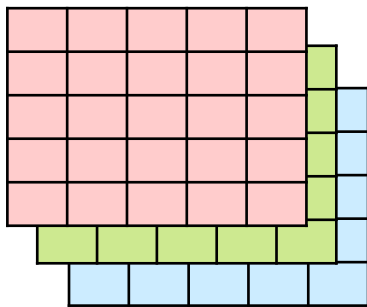
트랜스포머(Transformer), 2017년 등장

디퓨전 모델(DDPM, Denoising Diffusion Probabilistic Model), 2020년 등장

딥러닝의 발전



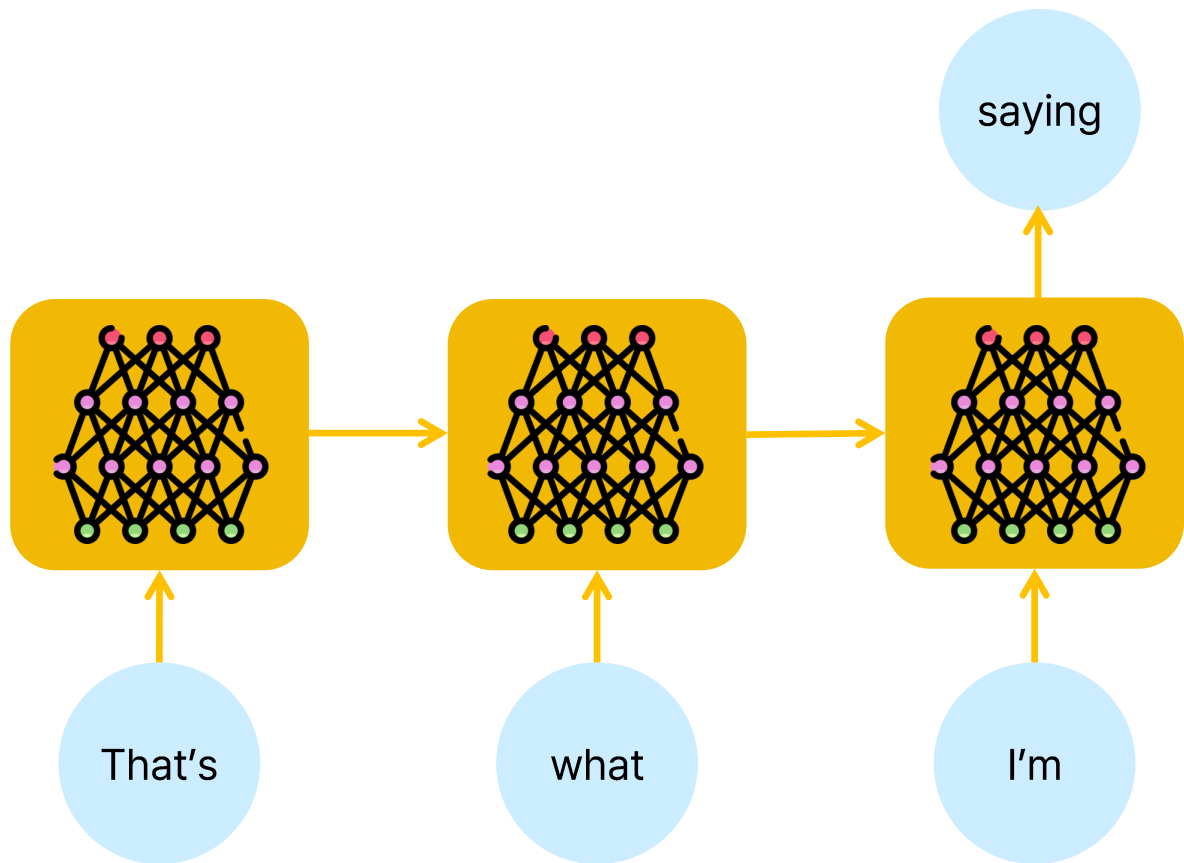
합성곱 신경망(CNN, Convolutional Neural Network)



	예측
새	0.92
강아지	0.05
고양이	0.01

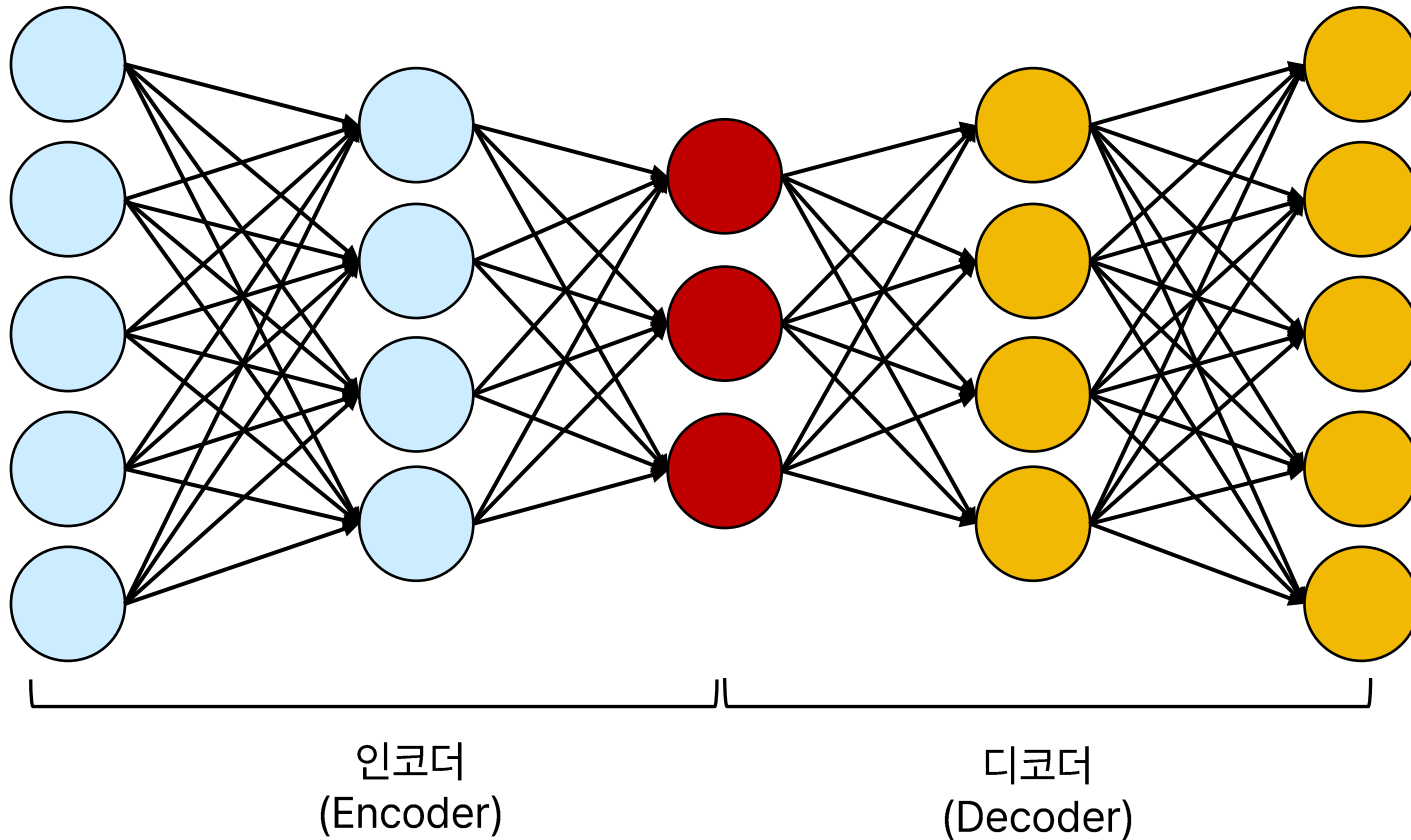
- 이미지, 영상 등 공간 정보 처리
- 이미지/영상 분류, 객체 검출 등에 활용

순환 신경망(RNN, Recurrent Neural Network)



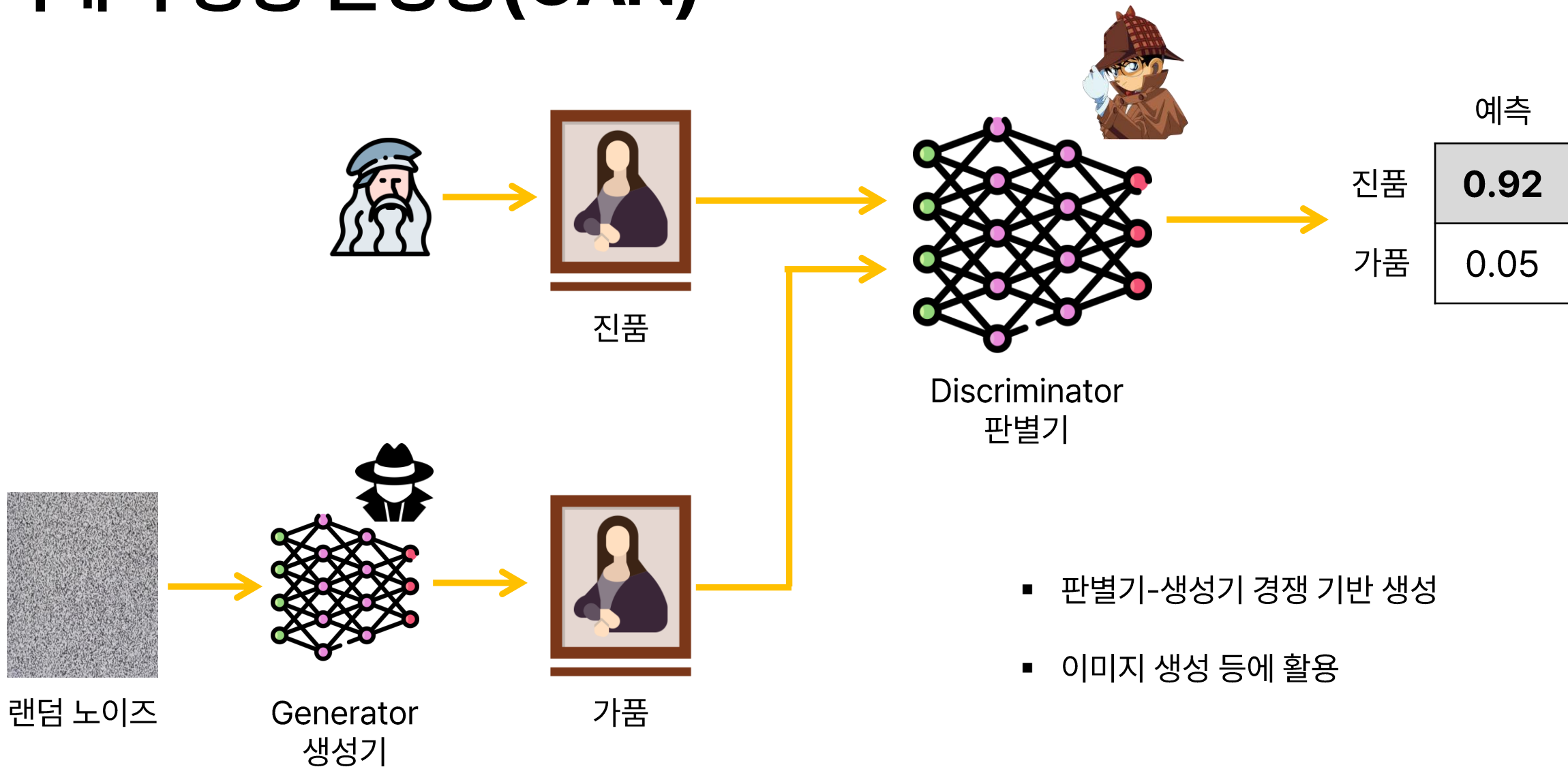
- 시퀀스 데이터 처리
- 텍스트 생성, 음성 인식, 시계열 예측 등에 활용

오토인코더(Auto Encoder)

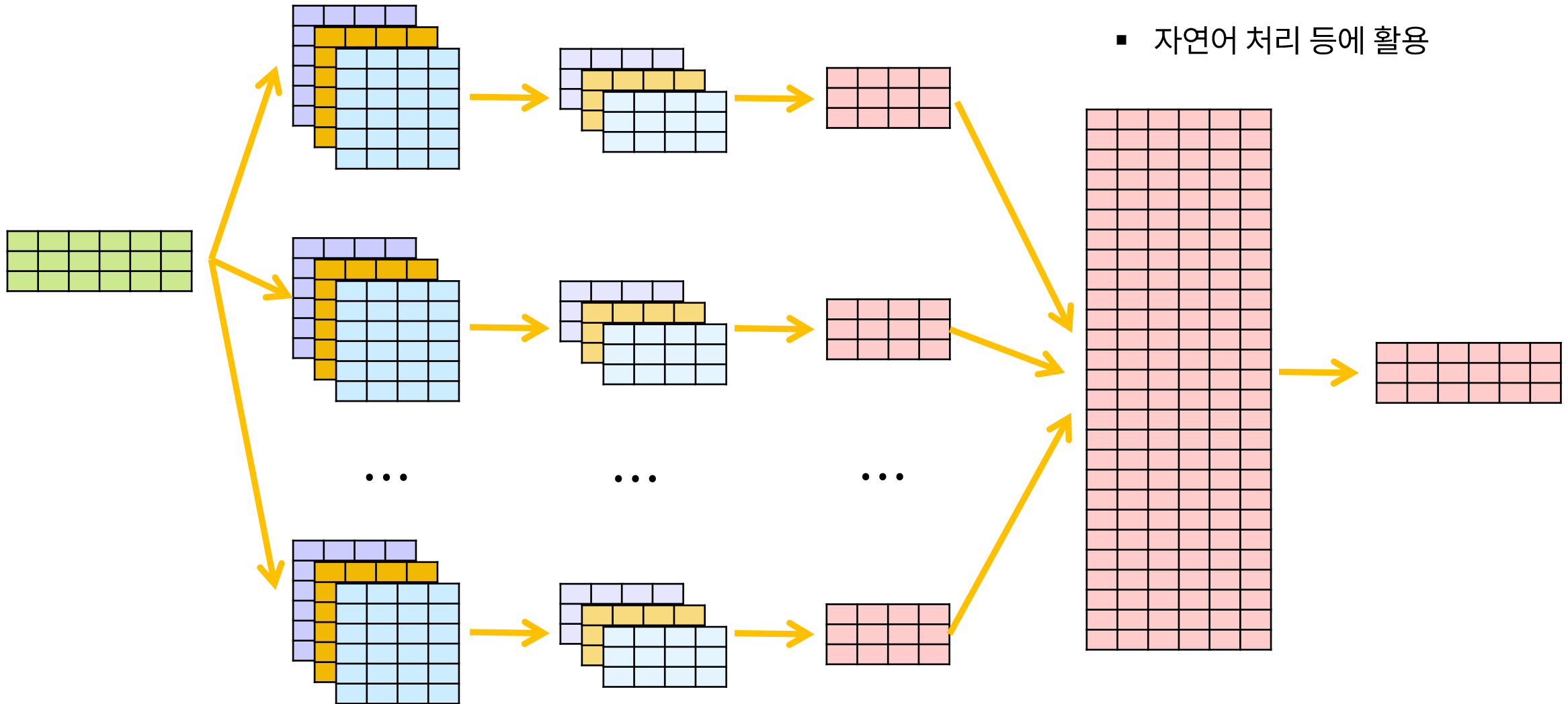


- 비지도학습, 차원축소
- 이상치 탐지, 데이터 압축, 추천 시스템 등에 활용

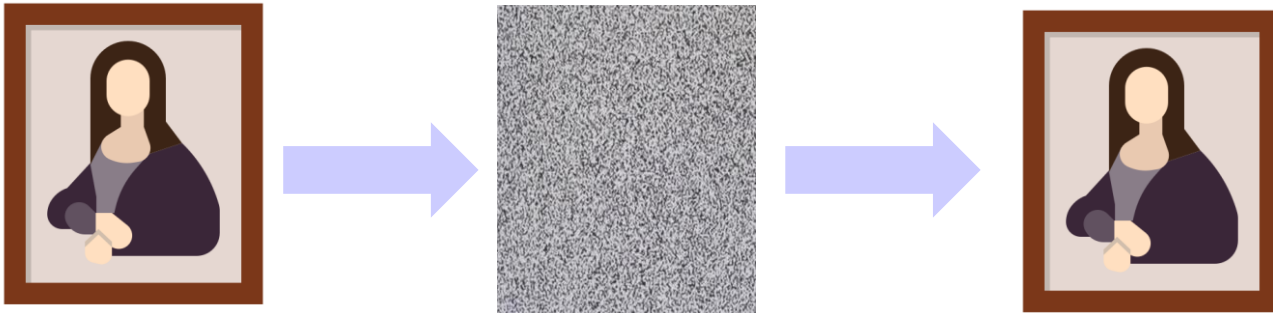
적대적 생성 신경망(GAN)



트랜스포머(Transformer)



디퓨전 모델(Diffusion model)



- 노이즈 제거 방식의 확률적 생성
- 이미지/영상/텍스트 생성
- 텍스트 생성은 효율이 떨어짐

감사합니다.

Q & A